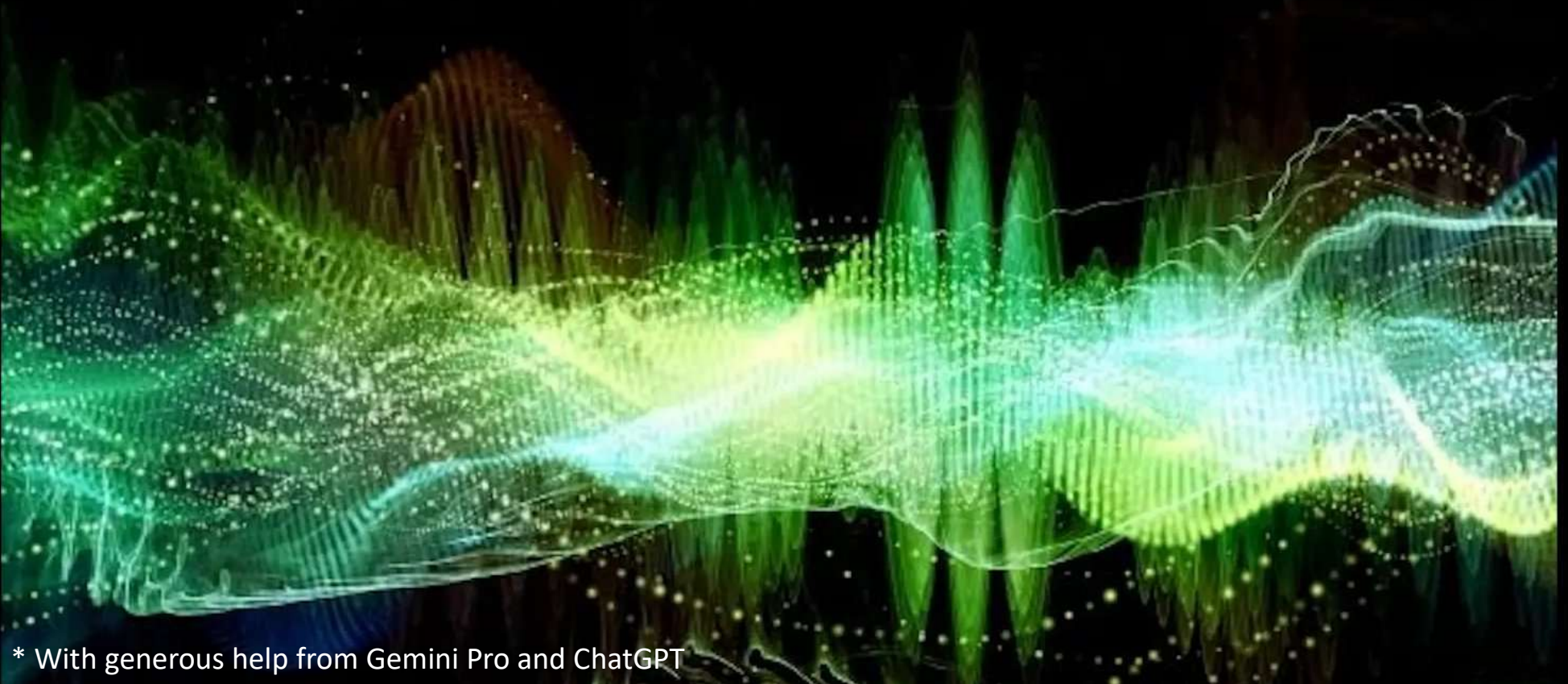


ECEN 4/5632 Theory and Application of Digital Filters

Lecture 11, 10-01-26

Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

WEEK 6 ● LECTURE 11

The DFT and Fast Convolution

From frequency samples to efficient streaming FIR filtering

Oppenheim & Schaffer • DFT / FFT / STFT | Raschka • CNN architecture and feature pipelines

Learning objectives

ROADMAP

- Interpret the N-point DFT as samples of the DTFT of a finite record.
- Distinguish linear convolution from N-point circular convolution.
- Choose zero-padding so DFT multiplication produces linear convolution. OLA OLS
- Explain and implement overlap-add and overlap-save block filtering.
- Connect FFT block processing to real-time preprocessing before ML inference.

Need to break-up long inputs into blocks need to preserve memory for filtering

For freq domain processing we need to take DFT/FFT of blocks

Sampling DTFT in frequency domain

0:00-0:03

Core question

How can an operation that is naturally defined for a whole sequence be implemented efficiently on an endless stream?

$$H(e^{j\omega}) = \sum_n h[n] e^{-j\omega n}$$

time/resource consuming to compute for real-life audio signals

DFT $H[k] = H(e^{j\omega})|_{\omega = \frac{2\pi k}{N}}$
N: block length

DFT review: a finite set of frequency samples

DFT

0:03–0:09

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1 \quad \text{DFT}$$

- The DFT is exactly the DTFT of the N-sample finite record evaluated at $\omega_k = 2\pi k/N$.
- The inverse DFT reconstructs those N time samples exactly.
- The DFT assumes an N-periodic extension when we interpret algebra such as convolution.

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \quad k = 0, 1, \dots, N-1 \quad \text{DFT blocklength } N$$

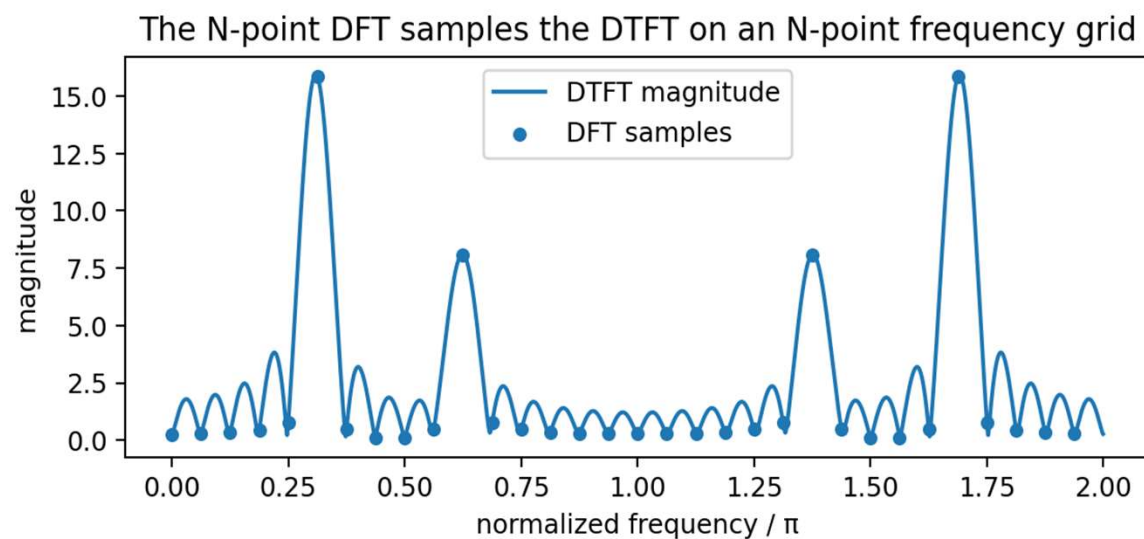
Do not confuse

A larger FFT can sample the same finite-record DTFT more densely, but zero-padding does not create new time-domain information.

The DFT samples the DTFT

DFT

0:09–0:14



Reading the figure

The smooth curve is the finite record's DTFT. The dots are the 32 DFT samples. An FFT is an algorithm for computing these samples efficiently.

Frequency grid: $\omega_k = 2\pi k/N$

Why the FFT matters computationally

DFT

0:14–0:20

Direct DFT: approximately $O(N^2)$ operations

FFT: approximately $O(N \log_2 N)$ operations

- This difference becomes decisive for long FIR filters and repeated streaming blocks.
- The FFT does not change the transform—it changes the computational algorithm.
- Fast convolution exploits the convolution theorem plus the FFT.

Example

For $N = 4096$, $N^2 \approx 16.8$ million while $N \log_2 N \approx 49$ thousand scale operations (constant factors omitted).

Linear convolution versus circular convolution

CONVOLUTION

0:20–0:27

Linear: $y[n] = \sum_m x[m] h[n-m]$

N-point circular: $y_N[n] = \sum_{m=0}^{N-1} x[m] h[(n-m) \bmod N]$

Key issue

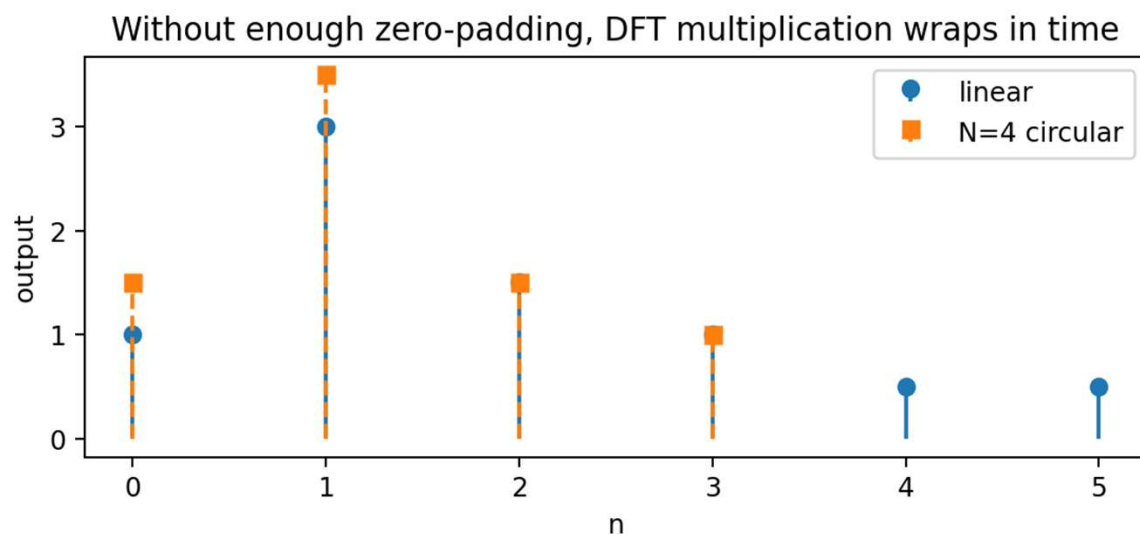
If N is too small, the tail of the linear convolution wraps onto the beginning.

- Linear convolution has length $L_x + L_h - 1$ for finite sequences.
- Circular convolution wraps samples modulo N .
- DFT multiplication corresponds to circular convolution of the N -point sequences.

Wrap-around is the error

CONVOLUTION

0:27–0:32



What happened?

The N=4 circular result folds the samples that belong at $n \geq 4$ back into $n = 0, 1, \dots$.

Circular is not “wrong”—it is a different operation.

Zero-padding makes DFT convolution linear

CONVOLUTION

0:32–0:38

Choose an FFT length satisfying

$$N_{\text{FFT}} \geq L_x + L_h - 1$$

- Pad both $x[n]$ and $h[n]$ to N_{FFT} .
- Multiply their N_{FFT} -point DFTs.
- Take the inverse DFT.
- The first $L_x + L_h - 1$ samples equal the linear convolution.

Example

$L_x = 1000$ and $L_h = 129$ require at least 1128 points. In practice, 2048 may be chosen for FFT efficiency.

Concept check: padding requirement

CONCEPT CHECK

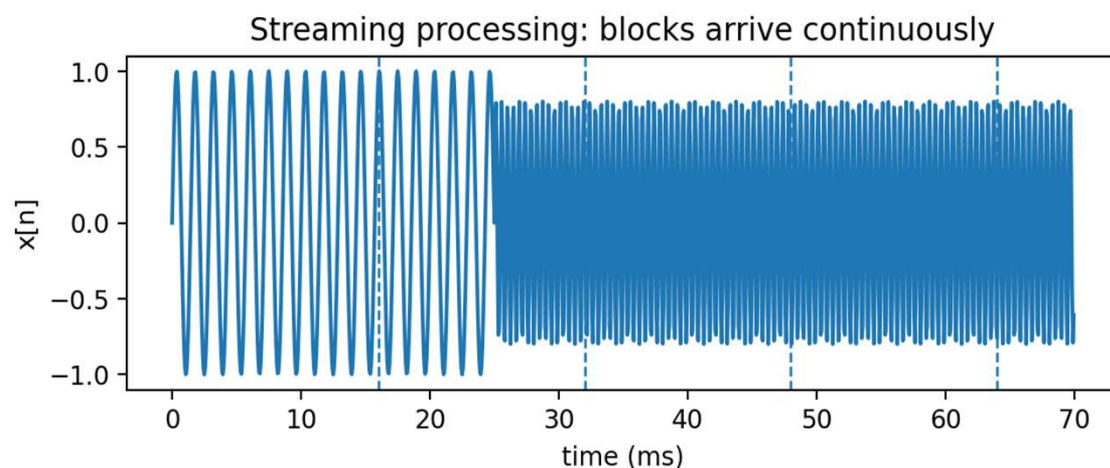
0:38–0:40

$x[n]$ has length 80 and $h[n]$ has length 33. What is the minimum FFT size for one-shot FFT-based linear convolution?

Why block processing is needed

STREAMING

0:40–0:46



- A microphone, sensor, or radio stream does not end so that we can take one giant FFT.
- We process finite blocks as they arrive.
- Each block must still reproduce the same linear FIR filtering result.
- Block length affects throughput, latency, memory, and FFT efficiency.

ML connection

Real-time inference often begins with streaming DSP: filtering, resampling, denoising, framing, and feature extraction.

Overlap-Add: filter independent input blocks

OVERLAP-ADD

0:46–0:54

1. Split $x[n]$ into non-overlapping blocks of B samples.
2. Zero-pad each block and $h[n]$ so $N_{\text{FFT}} \geq B + L_h - 1$.
3. FFT $\rightarrow$ multiply by $H[k] \rightarrow$ IFFT.
4. Add the overlapping output tails.

Each block produces $B + L_h - 1$ valid linear-convolution samples.

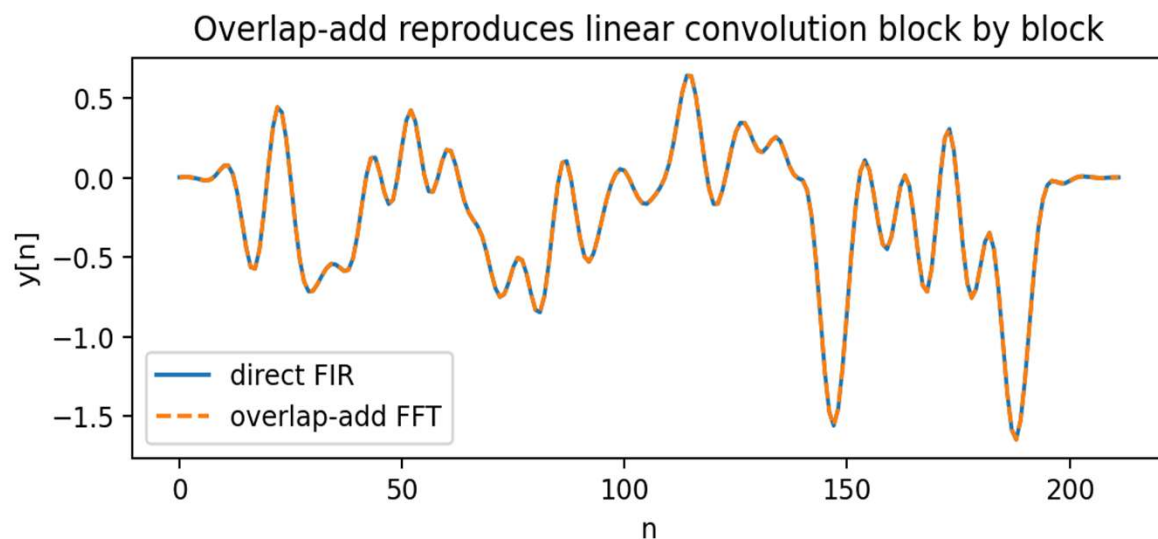
Why “add”?

Adjacent block convolutions overlap in time because the FIR has memory. Those overlapping contributions must be summed.

Overlap-Add gives the same FIR output

OVERLAP-ADD

0:54–0:58



Verification

The direct time-domain FIR and FFT overlap-add traces lie on top of one another to numerical precision.

This equivalence should be a coding unit test.

Overlap-Save: overlap the input, discard invalid outputs

OVERLAP-SAVE

0:58–1:06

- Choose an FFT size N_{FFT} and FIR length L_h .
- Each input block contains $L_h - 1$ old samples plus new samples.
- Perform N_{FFT} -point circular convolution.
- Discard the first $L_h - 1$ output samples, which contain time-aliasing contamination.
- Keep the remaining $N_{FFT} - L_h + 1$ valid samples.

Why “save”?

We save only the uncontaminated portion of each circular-convolution block and concatenate those portions.

Contrast

OLA overlaps and adds outputs. OLS overlaps inputs and discards corrupted outputs.

OLA versus OLS: engineering trade-offs

STREAMING

1:06–1:10

Overlap-Add

- Simple conceptual decomposition.
- Needs an output accumulation buffer.
- Input blocks are non-overlapping.

Overlap-Save

- Input blocks overlap by $L_h - 1$.
- No output addition step.
- Discard first $L_h - 1$ samples per block.

Both

Reuse $H[k]$ for every block and can be much faster than direct convolution for long FIR filters.

Choosing FFT and block sizes

STREAMING

1:10–1:13

- Larger FFTs often improve arithmetic efficiency per sample but increase buffering latency.
- Short blocks reduce latency but increase FFT overhead.
- Choose NFFT using measured platform performance, not only asymptotic complexity.
- On an edge device, memory movement and cache behavior can matter as much as multiply counts.

Design question

A classifier needs 10-ms latency. Would a huge “efficient” FFT block still be acceptable? No—the end-to-end latency constraint dominates.

ML pipeline connection

ML BRIDGE

1:13–1:14

stream → fast FIR / resampling → framing → features → model

- Fast convolution lets expensive preprocessing run continuously without changing the filter response.
- The same block boundaries should not create artifacts that leak into ML features.
- Streaming DSP and streaming inference must share latency and buffer budgets.

Think like a system designer

Accuracy, throughput, latency, and numerical equivalence all matter.

Lecture 11 wrap-up

WRAP-UP

1:14–1:15

- The DFT samples the finite-record DTFT.
- DFT multiplication gives circular convolution.
- Zero-padding prevents wrap-around and recovers linear convolution.
- Overlap-add and overlap-save extend FFT convolution to endless streams.
- Block size is both a DSP and systems-design parameter.

Exit ticket

Why is $N_{FFT} \geq B + L_h - 1$ required in overlap-add, and what fails if N_{FFT} is smaller?